Sorting Algorithms

Made Easy

Bubble · Insertion · Shell · Merge · Quick · Heap · Radix

Plain-language explanations, step-by-step traces, diagrams and C code for every method in the syllabus.

How every chapter is organised

- **The big idea** – the whole algorithm in two or three sentences, plus a real-life analogy.
- **Step-by-step rules** – what the algorithm does, in numbered steps.
- **Worked example** – a coloured trace so you can watch the array change.
- **C code** – a short, clean implementation you can write in the exam.
- **Complexity and properties** – time, space, stable, in-place, and *why* the numbers come out that way.
- **When to use it** – strengths, weaknesses and exam tips.

Contents

Part	Topic
0	Sorting basics, key terms and Big-O in 30 seconds
1	Bubble Sort
2	Insertion Sort
3	Shell Sort
4	Merge Sort
5	Quick Sort
6	Heap Sort
7	Radix Sort
8	Comparison table, cheat sheet and likely exam questions

0 Sorting Basics

Everything you need to understand before the seven algorithms

Sorting means arranging items in a fixed order – usually smallest to largest (ascending) or largest to smallest (descending). It matters because sorted data is much easier to work with: you can search it quickly with binary search, find duplicates, merge lists and produce rankings. Think of a dictionary – it is only usable because the words are sorted.

Words you must know

Term	Meaning	Tiny example
Pass	One full sweep over the data (or over the unsorted part).	Bubble sort's first pass pushes the largest value to the end.
Comparison-based	Decides the order only by comparing two elements at a time.	Bubble, insertion, shell, merge, quick, heap. (Radix is **not**.)
In-place	Sorts using only a constant amount of extra memory, O(1).	Bubble, insertion, heap sort rearrange the same array.
Stable	Equal elements keep their original relative order.	Sorting students by marks: two students with 80 stay in the same order as before.
Adaptive	Runs faster if the input is already nearly sorted.	Insertion sort on [1,2,3,5,4] needs very few moves.
Divide and conquer	Split the problem, solve the parts, combine the answers.	Merge sort and quick sort.
Internal vs external	Internal: all data fits in RAM. External: data is on disk and is sorted in chunks.	Merge sort is the standard choice for external sorting.

Big-O in 30 seconds

Big-O tells you how the running time *grows* when the number of items **n** grows. Do not worry about exact seconds – compare the growth. Suppose n = 1000:

Growth	Name	Steps for n = 1000	Which sorts?
O(n)	Linear	about 1,000	Best case of bubble and insertion
O(n log n)	Linearithmic	about 10,000	Merge, heap, quick (average)
$O(n^2)$	Quadratic	about 1,000,000	Bubble, insertion (average and worst)

Why O(n log n) is the magic number

Any sorting method that works **only by comparing elements** needs at least about n log n comparisons in the worst case. That is why merge, heap and quick sort are considered the "fast" comparison sorts. Radix sort escapes this limit only because it does *not* compare whole elements – it looks at digits.

Roadmap of the seven methods

Method	One-line idea	Family
Bubble	Swap neighbours that are out of order; big values float to the end.	Simple, $O(n^2)$
Insertion	Take the next item and slide it into the sorted part on the left.	Simple, $O(n^2)$
Shell	Insertion sort on items far apart first, then closer and closer.	Improved insertion
Merge	Split in halves, sort each, then merge two sorted halves.	Divide and conquer
Quick	Pick a pivot; smaller values go left, larger go right; repeat.	Divide and conquer
Heap	Build a max-heap, then keep removing the biggest element.	Selection with a heap
Radix	Sort digit by digit, from the last digit to the first.	Non-comparison

1 Bubble Sort

Compare neighbours, swap if wrong – the biggest value bubbles to the end

The big idea

Walk through the array from left to right. Each time, look at two **neighbouring** elements. If the left one is bigger than the right one, swap them. When you reach the end, the **largest** element has been carried all the way to the last position – just like an air bubble rising to the surface of water. Then repeat the walk for the remaining unsorted part.

> **Real-life analogy**
> Students stand in a line and the teacher walks along it. Whenever a taller student stands before a shorter one, the teacher swaps them. After one walk the tallest student is at the end of the line. After a few walks the whole line is in height order.

Step-by-step rules

1. Start at the first element. Compare **arr[j]** with **arr[j+1]**.
2. If arr[j] > arr[j+1], swap them. Otherwise leave them.
3. Move one step right and repeat until you reach the end of the unsorted part. This is **one pass**; the largest remaining value is now in its final place.
4. Repeat passes. Each pass can stop one position earlier because the tail is already sorted.
5. **Optimisation:** if a whole pass makes **no swap**, the array is sorted – stop early.

Worked example: 5, 1, 4, 2, 8



Figure 1 – Bubble sort on five numbers. Notice how 8 is already at the end after pass 1.

C code

C code

```c
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {  // number of passes
        int swapped = 0;  // optimisation flag
        for (int j = 0; j < n - i - 1; j++) {  // tail is already sorted
            if (arr[j] > arr[j + 1]) {  // neighbours out of order?
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        if (swapped == 0) break;  // no swap => sorted
    }
}
```

Complexity and properties

Best case	Average	Worst case	Extra space	Stable?	In-place?
O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes

- **Why $O(n^2)$?** There are about n passes and each pass makes up to n comparisons: (n−1) + (n−2) + … + 1 = n(n−1)/2 comparisons.
- **Why best case O(n)?** On an already sorted array the first pass makes no swap, the flag stops the loop, and only n−1 comparisons were done. (Without the flag the best case is still $O(n^2)$.)

- **Stable** because we swap only when arr[j] is *strictly* greater – equal elements are never swapped.

> **When to use it / exam tip**
> Bubble sort is easy to write and understand, but it is slow – it is used for teaching and for tiny or nearly sorted lists. Remember: **after pass k, the k largest elements are in their final positions.** Many exam questions ask for the array "after the 2nd pass".

2 Insertion Sort

Grow a sorted section by inserting one new item at a time

The big idea

Divide the array into two parts: a **sorted part** on the left (at the start it holds just the first element) and an **unsorted part** on the right. Take the first element of the unsorted part – call it the **key** – and slide it left into the correct place inside the sorted part, shifting bigger elements one step to the right to make room.

> **Real-life analogy**
> This is exactly how most people sort playing cards in their hand. You pick up one new card at a time and slide it into the right spot among the cards you already hold, which are already in order.

Step-by-step rules

1. Treat arr[0] as a sorted section of size 1.
2. For i = 1 to n−1: store arr[i] in **key**.
3. Look at the sorted section from right to left. While the element is **greater than key**, shift it one place to the right.
4. Put key into the gap that was created.
5. After the last element is inserted, the whole array is sorted.

Worked example: 5, 2, 4, 6, 1, 3



Figure 2 – The green block grows by one element every step.

C code

C code

```c
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];  // element to insert
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];  // shift bigger element right
            j--;
        }
        arr[j + 1] = key;  // drop key into the gap
    }
}
```

Complexity and properties

Best case	Average	Worst case	Extra space	Stable?	In-place?
O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes

- **Best case O(n):** if the array is already sorted, the while loop never runs, so each element costs just one comparison.
- **Worst case $O(n^2)$:** a reverse-sorted array – each key must travel all the way to the front, giving 1 + 2 + … + (n−1) shifts.
- **Adaptive and stable:** it stops shifting as soon as it finds an element ≤ key, so equal elements never cross each other.
- **Online:** it can sort data as it arrives, one element at a time.

When to use it / exam tip

Excellent for **small arrays** and **nearly sorted data**. Real libraries (for example the hybrid sorts inside many languages) use insertion sort for small chunks. Bubble and insertion are both $O(n^2)$ but insertion sort usually does fewer operations because it *shifts* instead of swapping repeatedly.

3 Shell Sort

Insertion sort with long jumps – fix far-apart items first

The big idea

Insertion sort is slow when a small value sits far to the right, because it can only move **one position at a time**. Shell sort fixes this. It first runs insertion sort on elements that are **gap** positions apart (for example every 4th element), then reduces the gap (4 → 2 → 1) and repeats. Early passes move elements across large distances; by the time the gap is 1 the array is *almost sorted*, and a normal insertion sort finishes very quickly.

> **Real-life analogy**
> Imagine tidying a messy bookshelf. First you swap books that are far from their correct section (big moves). Then you fine-tune neighbouring books (small moves). Rough sorting first, fine sorting last.

Step-by-step rules

1. Choose a starting gap, usually **n/2**.
2. For this gap, treat elements gap apart as separate small lists (e.g. gap 4 → positions 0,4 / 1,5 / 2,6 / 3,7) and sort each list with insertion sort. The array is now called **gap-sorted**.
3. Halve the gap (gap = gap/2) and repeat step 2.
4. Continue until gap = 1. The last pass is an ordinary insertion sort on a nearly sorted array – very fast.

Worked example: 35, 33, 42, 10, 14, 19, 27, 44 (n = 8)

The gaps used are 4, 2, 1. In the first row, items with the *same colour* belong to the same gap-4 list and are insertion-sorted among themselves.

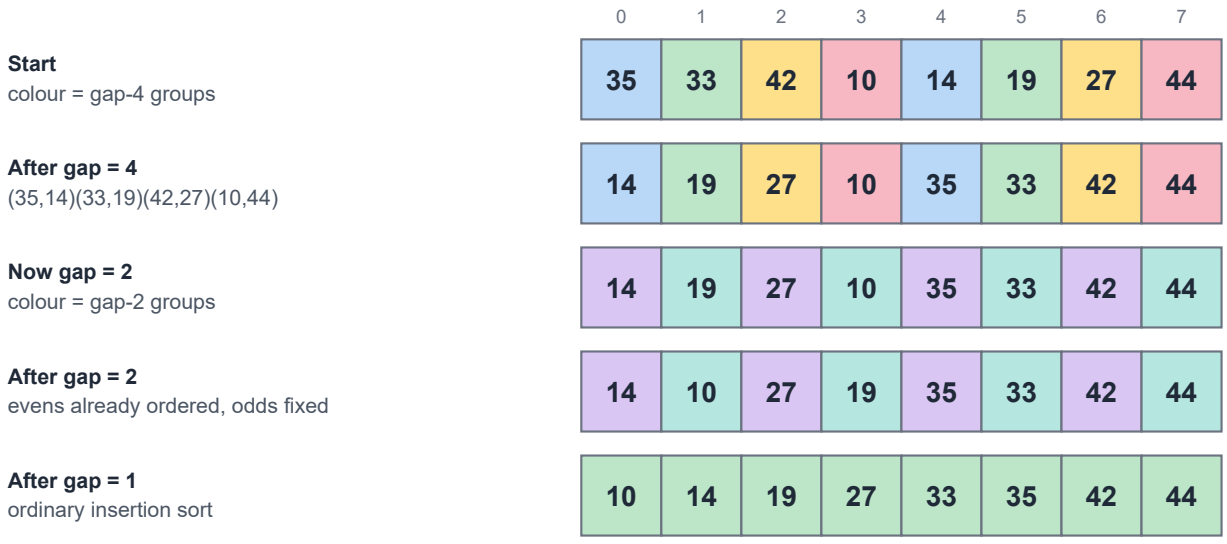


Figure 3 – Shell sort with gaps 4, 2, 1. After the gap-4 and gap-2 passes the array is already close to sorted, so the final pass moves things only a short distance.

C code

C code

```
void shellSort(int arr[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {  // shrink the gap
        for (int i = gap; i < n; i++) {  // insertion sort with step = gap
            int temp = arr[i];
            int j = i;
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];  // shift by gap, not by 1
                j -= gap;
            }
            arr[j] = temp;
        }
    }
}
```

Gap sequences – the speed depends on them

Sequence	Gaps (example)	Worst-case time
Shell's original (n/2, n/4, … 1)	8, 4, 2, 1	$O(n^2)$
Hibbard (2^k − 1)	1, 3, 7, 15, 31 …	$O(n^{1.5})$
Knuth ((3^k − 1)/2)	1, 4, 13, 40, 121 …	$O(n^{1.5})$
Sedgewick	1, 5, 19, 41, 109 …	$O(n^{4/3})$

Complexity and properties

Best case	Average	Worst case	Extra space	Stable?	In-place?
O(n log n)	**≈ $O(n^{1.5})$**	**$O(n^2)$**	**O(1)**	**No**	**Yes**

- **Average case** depends on the gap sequence; with good sequences it is far better than $O(n^2)$ but there is no single simple formula.
- **Not stable:** moving an element by a large gap can jump it over an equal element.
- **In-place and simple:** no recursion and no extra array – the reason Shell sort is popular in embedded systems.

When to use it / exam tip

Shell sort = **insertion sort + gaps**. A very common exam question is "how does Shell sort improve insertion sort?" – answer: it allows elements to move long distances early, so the final gap-1 pass has very little work to do.

4 Merge Sort

Divide the array in halves, sort each half, then merge

The big idea

Merge sort is a **divide and conquer** algorithm. A list of one element is already sorted, and **two sorted lists can be merged into one sorted list very cheaply**. So: keep splitting the array in half until every piece has one element, then keep merging pieces back together in sorted order.

> **Real-life analogy**
> Two friends each hold a sorted pile of exam papers. To combine them, repeatedly compare the top paper of each pile and place the smaller one on a new pile. When one pile runs out, put the rest of the other pile on top. That is the **merge** step.

Step-by-step rules

1. **Divide:** find the middle, split the array into a left half and a right half.
2. **Conquer:** recursively sort the left half and the right half.
3. **Combine:** merge the two sorted halves into one sorted array using the merge procedure.
4. **Base case:** an array with 0 or 1 element is already sorted – stop splitting.

Worked example: 38, 27, 43, 3, 9, 82, 10

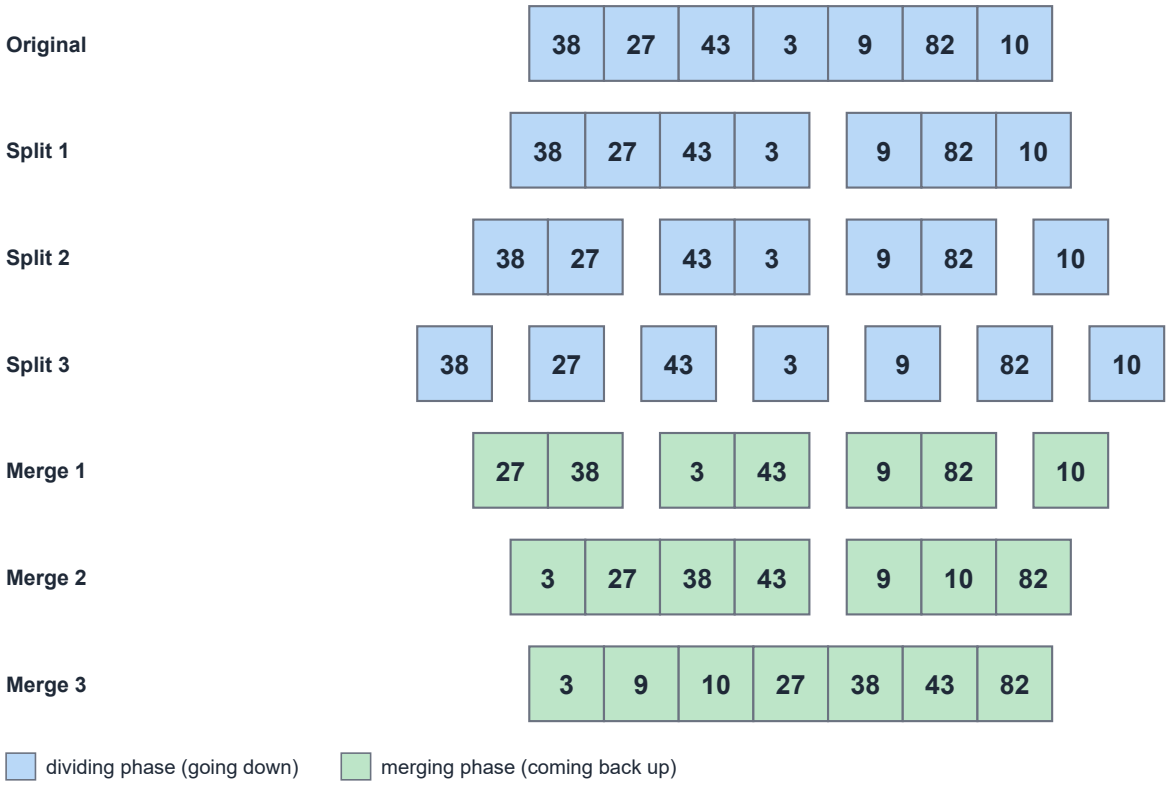


Figure 4 – The recursion tree. Each group is a sub-array. The array is cut in half until single elements remain, then merged upward in sorted order.

The merge step in detail

Merge Left = [3, 27, 38, 43] and Right = [9, 10, 82]. Keep one pointer in each list; always copy the smaller front value into the output.

Step	Compare	Smaller → copied	Output so far
1	3 vs 9	3 (left)	3
2	27 vs 9	9 (right)	3, 9
3	27 vs 10	10 (right)	3, 9, 10
4	27 vs 82	27 (left)	3, 9, 10, 27
5	38 vs 82	38 (left)	3, 9, 10, 27, 38
6	43 vs 82	43 (left)	3, 9, 10, 27, 38, 43
7	left list empty	copy remaining 82	3, 9, 10, 27, 38, 43, 82

C code

C code

```c
#include <stdlib.h>

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int *L = (int *)malloc(n1 * sizeof(int));  // temporary copies
    int *R = (int *)malloc(n2 * sizeof(int));
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {  // pick the smaller front value
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];  // copy leftovers
    while (j < n2) arr[k++] = R[j++];
    free(L); free(R);
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;  // middle index
        mergeSort(arr, l, m);  // sort left half
        mergeSort(arr, m + 1, r);  // sort right half
        merge(arr, l, m, r);  // combine
    }
}
```

Complexity and properties

Best case	Average	Worst case	Extra space	Stable?	In-place?
O(n log n)	**O(n log n)**	**O(n log n)**	**O(n)**	**Yes**	**No**

- **Why n log n?** Halving n repeatedly gives about $\log_2 n$ levels. At every level the merge steps together touch all n elements once. So the work is n × log n – in *every* case, because the splitting never depends on the data. Recurrence: T(n) = 2T(n/2) + n.

- **Extra space O(n):** the temporary arrays L and R are needed during merging, so merge sort is *not* in-place.
- **Stable:** using L[i] <= R[j] (not <) takes the left element first when values are equal.

> **When to use it / exam tip**
> Guaranteed O(n log n) and stable – great for **linked lists** and **external sorting** (huge files on disk). Its drawback is the extra memory. Exam favourites: draw the recursion tree, show the merge of two lists, and state T(n) = 2T(n/2) + n.

5 Quick Sort

Choose a pivot, partition around it, repeat on both sides

The big idea

Quick sort is also **divide and conquer**, but the hard work is done *before* the recursion instead of after. Pick one element as the **pivot**. Rearrange the array (this is called **partitioning**) so that everything smaller than the pivot is on its left and everything larger is on its right. The pivot is now in its **final sorted position**. Then sort the left part and the right part the same way.

> **Real-life analogy**
> Line students up by height. Pick one student as the reference. Everyone shorter goes to the left of the reference, everyone taller goes to the right. The reference student is now in the correct place for good. Repeat the idea separately for the left group and the right group.

Step-by-step rules (Lomuto partition, pivot = last element)

1. Choose the pivot = last element of the current range. Keep a marker **i** just before the range.
2. Scan j from the start of the range up to (but not including) the pivot.
3. If arr[j] ≤ pivot: move marker i one step right and **swap arr[i] with arr[j]**. This grows the "small values" zone.
4. After the scan, swap the pivot with arr[i+1]. The pivot is now in its final place; return its index.
5. Recursively quick-sort the part left of the pivot and the part right of the pivot.

Worked example: 8, 3, 1, 7, 0, 10, 2

First partition – pivot is the last element, **2**. Only the scans that cause a swap are shown (for 8, 3, 7 and 10 nothing happens because they are bigger than 2).

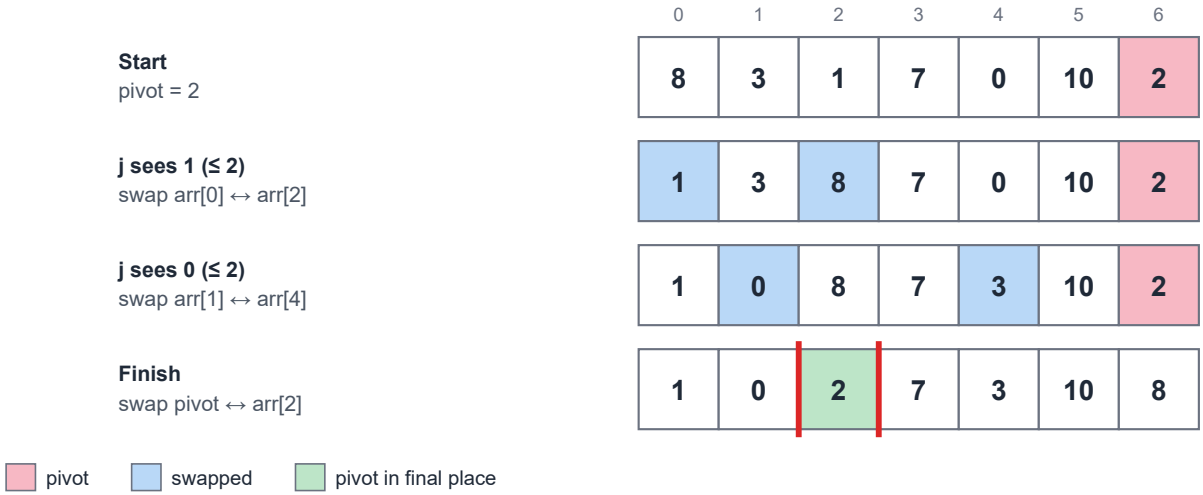


Figure 5 – First partition. The red bars mark the pivot's final position: smaller values on its left, larger on its right.

Recursion on the two parts – each part is partitioned the same way:

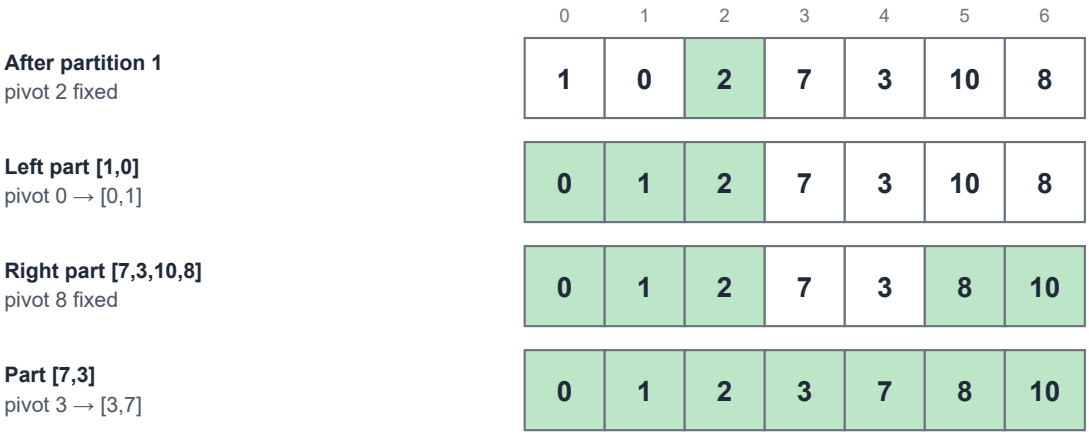


Figure 6 – Every green box is a value that has reached its final position. The array is sorted when all boxes are green.

C code

C code

```c
void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];  // pivot = last element
    int i = low - 1;  // boundary of the "small" zone
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);  // move small value into the zone
        }
    }
    swap(&arr[i + 1], &arr[high]);  // put pivot in its final place
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int p = partition(arr, low, high);
        quickSort(arr, low, p - 1);  // left of pivot
        quickSort(arr, p + 1, high);  // right of pivot
    }
}
```

Complexity and properties

Best case	Average	Worst case	Extra space	Stable?	In-place?
O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No	Yes

- **Best / average case:** the pivot splits the array into two roughly equal parts, giving log n levels with n work each → O(n log n).
- **Worst case $O(n^2)$:** the pivot is always the smallest or largest element (for example, an already sorted array with the last element as pivot). One side is empty, the other has n−1 elements, so we get n levels with up to n work each.
- **Extra space O(log n):** only the recursion stack (O(n) in the worst case).
- **Not stable:** the long-distance swaps can change the order of equal elements.

Choosing the pivot

Strategy	Idea	Comment
First or last element	Simplest to code	Worst case on sorted or reverse-sorted input
Random element	Pick a random index and swap it to the end	Makes the worst case extremely unlikely
Median of three	Median of first, middle and last	Good protection on sorted input

When to use it / exam tip

In practice quick sort is usually the **fastest** general-purpose sort: it works in-place, has a small constant factor and is cache friendly. Remember the contrast with merge sort – **merge sort divides trivially and works hard when combining; quick sort works hard when dividing (partition) and combining costs nothing.**

6 Heap Sort

Build a max-heap, then repeatedly take out the largest element

First, what is a heap?

A **binary heap** is a **complete binary tree** (all levels full except possibly the last, which fills from the left) that obeys the **heap property**. In a **max-heap** every parent is greater than or equal to its children, so the **largest element is always at the root**. A heap is stored in a plain array – no pointers needed. For the element at index **i** (0-based):

Relation	Index formula	Example for node 5 at i = 1 in [10, 5, 3, 4, 1]
Parent	(i − 1) / 2	(1 − 1)/2 = 0 → value 10
Left child	2i + 1	2(1) + 1 = 3 → value 4
Right child	2i + 2	2(1) + 2 = 4 → value 1

The big idea

Heap sort uses the fact that the biggest value is always at the root. It works in two phases:

1. **Build a max-heap** from the unsorted array. Start from the last non-leaf node (index n/2 − 1) and move up to the root, fixing each node with **heapify**.
2. **Repeat n−1 times:** swap the root (current maximum) with the last element of the heap, shrink the heap size by 1 (that last position is now sorted), and **heapify the root** to restore the heap property.

> **Real-life analogy**
> Think of a tournament that always keeps the strongest player at the top of the bracket. Take the champion out and place them at the end of the results list, then re-run the bracket to find the next champion. Repeat until everyone is placed.

What is heapify (sift-down)?

Heapify fixes a single node whose children are already valid heaps: compare the node with its two children, and if a child is larger, **swap with the larger child**. Then continue the same check at the child's position, until the node is bigger than both children or becomes a leaf.

Worked example: 4, 10, 3, 5, 1

Phase 1 – build the heap. The last non-leaf node is index 1 (value 10); its children 5 and 1 are smaller, so nothing changes. Then at the root: 4 is smaller than its larger child 10, so swap; 4 now sits at index 1 where its children are 5 and 1, so swap with 5.

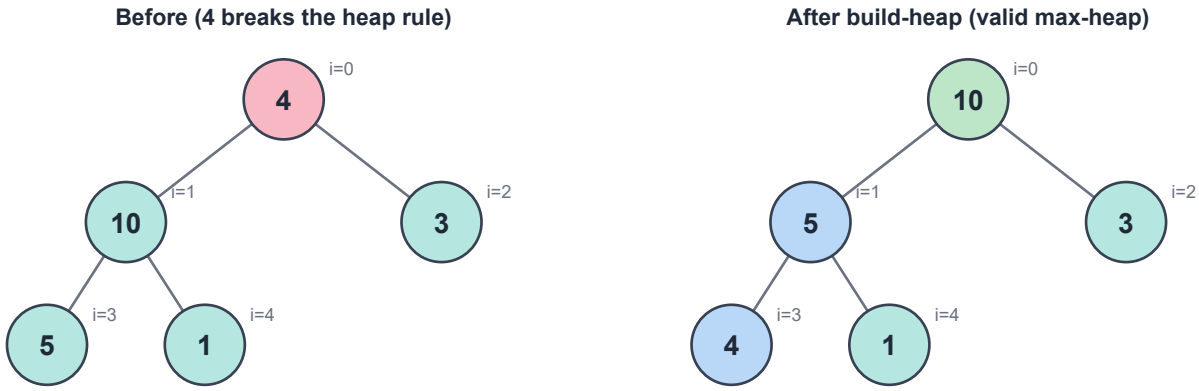


Figure 7 – Phase 1 as trees. Every parent is now at least as big as its children.

Phase 2 – sort. Green means "already in final position". The heap is the part on the left.

	0	1	2	3	4
Max-heap built	10	5	3	4	1
Swap root 10 ↔ last	1	5	3	4	10
Heapify (size 4)	5	4	3	1	10
Swap root 5 ↔ last	1	4	3	5	10
Heapify (size 3)	4	1	3	5	10
Swap root 4 ↔ last	3	1	4	5	10
Swap root 3 ↔ last	1	3	4	5	10

swapped root (after swap) sorted – final position

Figure 8 – The sorted region grows from the right while the heap shrinks on the left. (After the heap shrinks to size 2, [3,1] already obeys the heap rule, so heapify changes nothing.)

C code

C code

```c
void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}

void heapify(int arr[], int n, int i) {  // sift-down node i in a heap of size n
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) {  // a child is bigger: swap and continue
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)  // phase 1: build max-heap
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {  // phase 2: extract maximum
        swap(&arr[0], &arr[i]);  // move max to its final position
        heapify(arr, i, 0);  // restore heap on the smaller heap
    }
}
```

Complexity and properties

Best case	Average	Worst case	Extra space	Stable?	In-place?
O(n log n)	O(n log n)	O(n log n)	O(1)	No	Yes

- **Phase 1 costs O(n):** most nodes are near the bottom and sink very little. The total work of building a heap is linear.
- **Phase 2 costs O(n log n):** n−1 extractions, and each heapify walks down at most the height of the tree, which is log n.
- **Not stable:** swapping the root with the last element jumps equal values over each other.
- **Ascending order needs a max-heap** (biggest goes to the back). Use a min-heap if you want to repeatedly pull out the smallest instead.

When to use it / exam tip

Heap sort gives a **guaranteed O(n log n)** like merge sort *and* uses O(1) extra space like quick sort – the best of both on paper. In practice it is slower than quick sort because of poor cache behaviour. Heaps are also the basis of **priority queues**.

7 Radix Sort

Sort numbers digit by digit – no comparisons between numbers

The big idea

All the earlier methods compare two numbers directly. Radix sort never does. Instead it sorts the numbers by **one digit at a time**, starting from the **least significant digit** (units), then tens, then hundreds, and so on. For each digit it uses a **stable** sort (usually counting sort) that drops numbers into ten buckets 0–9 and collects them back in order.

Real-life analogy

A post office sorts letters by PIN code. First everything is put into 10 boxes by the last digit, the boxes are emptied in order 0 to 9, and the letters are re-sorted by the second-last digit, and so on. After the first digit has been processed everywhere, the letters are fully in order.

Step-by-step rules (LSD radix sort)

1. Find the largest number to know how many digits (**d**) there are.
2. For each digit position, starting at units: distribute the numbers into buckets 0–9 by that digit, then collect the buckets in order 0 → 9 – keeping the **original relative order** inside every bucket (stability).
3. Move to the next digit (tens, hundreds, …) and repeat.
4. After the last digit pass, the array is completely sorted.

Why must the digit-sort be stable?

When we sort by the tens digit, two numbers with the same tens digit must stay in the order the *units* pass gave them. Stability carries the earlier work forward; without it radix sort would give wrong answers.

Worked example: 170, 45, 75, 90, 802, 24, 2, 66

The largest number, 802, has 3 digits, so three passes are needed. The table shows which numbers fall into each bucket (in the order they arrive):

Bucket	Pass 1 – units digit	Pass 2 – tens digit	Pass 3 – hundreds digit
0	170, 90	802, 2	2, 24, 45, 66, 75, 90
1	–	–	170
2	802, 2	24	–
3	–	–	–
4	24	45	–
5	45, 75	–	–
6	66	66	–
7	–	170, 75	–
8	–	–	802
9	–	90	–

	0	1	2	3	4	5	6	7
Input	170	45	75	90	802	24	2	66
After pass 1 sorted by units digit	170	90	802	2	24	45	75	66
After pass 2 sorted by tens digit	802	2	24	45	66	170	75	90
After pass 3 sorted by hundreds digit	2	24	45	66	75	90	170	802

Figure 9 – The array after every pass. After pass 2 the numbers are correctly ordered by their last two digits; after pass 3 by all digits.

How counting sort does each pass

For one digit position: (1) **count** how many numbers have each digit 0–9; (2) turn the counts into **running totals** – each total says where that digit's group ends in the output; (3) walk the input from **right to left**, place each number at its position and decrease that count. Walking right to left is what keeps the sort stable.

C code

C code

```c
#include <stdlib.h>

int getMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > max) max = arr[i];
    return max;
}

void countSort(int arr[], int n, int exp) {  // stable sort on one digit
    int *output = (int *)malloc(n * sizeof(int));
    int count[10] = {0};
    for (int i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;  // 1) count digits
    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];  // 2) running totals
    for (int i = n - 1; i >= 0; i--) {  // 3) place, right to left
        int d = (arr[i] / exp) % 10;
        output[count[d] - 1] = arr[i];
        count[d]--;
    }
    for (int i = 0; i < n; i++) arr[i] = output[i];
    free(output);
}

void radixSort(int arr[], int n) {
    int max = getMax(arr, n);
    for (int exp = 1; max / exp > 0; exp *= 10)  // units, tens, hundreds ...
        countSort(arr, n, exp);
}
```

Complexity and properties

Best case	Average	Worst case	Extra space	Stable?	In-place?
O(d(n+k))	O(d(n+k))	O(d(n+k))	O(n+k)	Yes	No

- **Notation:** n = number of items, d = number of digits in the largest number, k = base (10 for decimal digits).
- **Why linear-like?** Each pass costs O(n + k) and there are d passes. If d is small and fixed, the running time is effectively O(n) – faster than the n log n comparison limit.
- **Not in-place:** needs an output array of size n plus the count array.
- **Limits:** works on integers or fixed-length keys (strings, dates). It is not suited to arbitrary real numbers or when d is very large compared with log n.

When to use it / exam tip

Use radix sort for **large lists of integers with a limited number of digits** – phone numbers, PIN codes, IDs. Exam keywords: **non-comparison**, **stable**, **LSD first**, **buckets 0–9**, O(d(n+k)).

8 Cheat Sheet and Exam Prep

Compare all seven methods at a glance

Master comparison table

Sort	Best	Average	Worst	Space	Stable	In-place	Technique
Bubble	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes	Swap neighbours
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes	Insert into sorted part
Shell	O(n log n)	$\approx O(n^{1.5})$	$O(n^2)$	O(1)	No	Yes	Gapped insertion
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	No	Divide and merge
Quick	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No	Yes	Pivot partition
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No	Yes	Heap + extract max
Radix	O(d(n+k))	O(d(n+k))	O(d(n+k))	O(n+k)	Yes	No	Digit buckets

Which one should I pick?

Situation	Best choice	Why
Very small array, or nearly sorted data	Insertion sort	Tiny overhead, O(n) on sorted input
General-purpose, speed matters most	Quick sort	Fastest on average, in-place
Need guaranteed O(n log n) and stability	Merge sort	Same speed in all cases, stable
Need guaranteed O(n log n) with almost no extra memory	Heap sort	O(1) space, no bad inputs
Medium arrays, no recursion allowed	Shell sort	Simple loops, much better than insertion
Many integers with few digits	Radix sort	Beats n log n by avoiding comparisons
Teaching or very small lists	Bubble sort	Simplest to understand and write

Memory tricks

- **Stable ones: B**ubble, **I**nsertion, **M**erge, **R**adix. **Unstable ones: S**hell, **Q**uick, **H**eap.
- **Always O(n log n):** merge and heap. **Has an $O(n^2)$ worst case but is fast on average:** quick (and shell with a poor gap sequence).
- **Needs extra array:** merge and radix. **Uses recursion stack:** merge and quick.
- **One line each:** Bubble – biggest bubbles up. Insertion – cards in hand. Shell – insertion with gaps. Merge – split then stitch. Quick – pivot then partition. Heap – build the tree, take the top. Radix – digit by digit.

Frequently asked exam questions (with short answers)

Question	Short answer
When is quick sort $O(n^2)$?	When the pivot is always the smallest or largest element, e.g. sorted input with first/last element as pivot. Random or median-of-three pivots avoid it.
Why does merge sort need extra space?	The merge step copies the two halves into temporary arrays so it can overwrite the original positions safely – O(n) extra.
Difference between insertion and Shell sort?	Insertion compares and shifts adjacent items only; Shell first compares items gap positions apart, so elements travel long distances early and the final pass has little work.
Why is building a heap O(n) and not O(n log n)?	Half of the nodes are leaves (no work) and most others sink only one or two levels, so the total work sums to a linear amount.
Can any sort beat O(n log n)?	Not by comparisons alone. Radix (and counting) sort beat it by using digit values instead of comparing elements.
Which sorts are best for an already sorted array?	Bubble (with flag) and insertion sort: O(n).
Which sort after the k-th pass has the k largest items in place?	Bubble sort.
Why must radix sort use a stable digit-sort?	To keep the order created by earlier (less significant) digit passes.

Appendix – a small test program (C)

Paste any one of the sorting functions from this guide above **main** and run it to test.

C code – test driver

```c
#include <stdio.h>

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {64, 25, 12, 22, 11, 90, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Before: "); printArray(arr, n);
    bubbleSort(arr, n);  // or insertionSort, shellSort, heapSort, radixSort
      // mergeSort(arr, 0, n - 1); // merge sort takes (arr, left, right)
      // quickSort(arr, 0, n - 1); // quick sort takes (arr, low, high)
    printf("After: "); printArray(arr, n);
    return 0;
}
```